

HarmonE

A Self-Adaptive Approach to Architecting
Sustainable MLOps

A Complete, Self-Contained Explanatory Document

Covering motivation, problem statement, background concepts,
architecture, algorithms, experiments, and results

Based on the paper by:

Hiya Bhatt Shaunak Biswas Srinivasan Rakhunathan Karthik Vaidhyanathan

IIIT Hyderabad, India Microsoft, India

arXiv:2505.13693v1 [cs.SE] — May 2025

This document is an in-depth explanation of the paper. No prior knowledge of MLOps,
self-adaptive systems, or sustainability engineering is assumed.

Contents

1	Overview: What Is This Paper About?	2
2	Background: Essential Concepts	2
2.1	Machine Learning Enabled Systems (MLS)	2
2.2	The Core Problem with Deployed ML Models	3
2.3	MLOps: Automating the ML Lifecycle	3
2.4	Sustainability as a Multi-Dimensional Concept	4
2.5	Self-Adaptive Systems and the MAPE-K Loop	4
2.6	KL Divergence: Measuring Data Shift	5
2.7	Exponential Moving Average (EMA)	5
3	Motivation: Why HarmonE Is Needed	5
3.1	The Gap in Existing Approaches	5
3.2	The Problem with Instantaneous Reactions	6
3.3	The Real-World Trigger: ITS Digital Twin	6
4	HarmonE: The Proposed Solution	7
4.1	Component 1: The Managed System	7
4.1.1	Training Subsystem	7
4.1.2	Inference Subsystem	7
4.2	Component 2: The Decision Map (Design-Time Artifact)	8
4.3	Component 3: The Managing System	8
4.3.1	The Knowledge Base	9
4.3.2	Monitor: Observing the System	9
4.3.3	Analyzer: Detecting Uncertainty	10
4.3.4	Planner: Selecting an Adaptation Strategy	11
4.3.5	Executor: Carrying Out the Adaptation	12
5	The Use Case: Traffic Flow Prediction in a Digital Twin	12
5.1	Setting	12
5.2	Model Spectrum	12
5.3	Drift Induction	13
6	Experimental Design and Baselines	13
6.1	Dataset Split	13
6.2	Baselines	13
6.3	Metrics	14

7	Results and Analysis	14
7.1	RQ1: Long-Term Environmental Sustainability	14
7.2	RQ2: Energy vs. Accuracy Trade-Off	15
7.3	RQ3: Efficiency of Adaptation Decisions	16
8	Threats to Validity	16
8.1	Internal Validity	16
8.2	Construct Validity	16
8.3	External Validity	17
9	Related Work: Where HarmonE Fits	17
10	The Big Picture: How Everything Connects	17
11	Conclusion and Future Directions	18
11.1	What Was Achieved	18
11.2	Future Work	18

1 Overview: What Is This Paper About?

This paper introduces **HarmonE**, a software architectural framework that makes Machine Learning (ML) systems smarter about managing themselves over time — particularly in balancing energy efficiency, cost, accuracy, and maintainability. Rather than simply deploying an ML model and hoping it continues to perform well, HarmonE watches the system continuously, detects problems like degradation or data shift, and takes corrective action automatically — all while keeping the system “green” (energy-efficient) and economically sustainable.

The paper is the work of researchers at IIIT Hyderabad (India) and Microsoft India, validated using a real-world traffic prediction scenario within a Digital Twin of an Intelligent Transportation System.

The One-Sentence Summary

HarmonE is an architecture that teaches an ML system to manage its own health — switching models, retraining only when truly needed, and reusing old knowledge — so it stays accurate, cheap, and energy-efficient over its entire operational lifetime.

2 Background: Essential Concepts

Before diving into HarmonE itself, we need to understand several foundational ideas. This section builds them from the ground up.

2.1 Machine Learning Enabled Systems (MLS)

A **Machine Learning Enabled System (MLS)** is any real-world software system whose core functionality depends on one or more ML models. Examples include:

- A traffic-control platform that predicts congestion using sensor data.
- A hospital system that predicts patient readmission risk.
- An industrial robot that uses a vision model to detect defects on a production line.

MLS are increasingly common. Their distinguishing characteristic is that their behavior is learned from data, not explicitly programmed. This makes them powerful — but also fragile, because the world changes while the model stays static.

2.2 The Core Problem with Deployed ML Models

Once an ML model is trained and deployed, the world does not stand still. Several things can go wrong:

1. **Data Drift:** The statistical distribution of incoming data changes. For a traffic predictor, this might happen because a new highway opens, changing the usual flow patterns.
2. **Model Drift / Concept Drift:** The relationship between inputs and outputs changes. What was true (heavy rain $\Rightarrow$ slow traffic) may change in unexpected ways over time.
3. **Performance Degradation:** As drift accumulates, the model's predictions become less accurate. The model has effectively become "stale."

The traditional fix is **retraining** — feeding the model fresh data and relearning the parameters. But retraining is expensive. The paper cites the striking example that retraining a large model like BERT can:

- Consume over **1,500 kWh** of electricity.
- Emit more than **700 lbs of CO₂**.
- Cost upwards of **\$12,000**.

Retraining frequently is therefore not just technically costly — it is environmentally and economically harmful.

2.3 MLOps: Automating the ML Lifecycle

MLOps (Machine Learning Operations) is the discipline of automating the end-to-end lifecycle of an ML model: data ingestion, preprocessing, training, validation, deployment, monitoring, and retraining. Think of it as DevOps applied to ML.

MLOps pipelines are designed to make ML deployment more reliable and repeatable. However, they have a significant blind spot: most MLOps pipelines are designed around a single dimension of quality — *technical* performance (accuracy, latency). They largely ignore:

- Environmental costs (energy, emissions).
- Economic costs (compute bills, storage overhead).
- Social concerns (user trust, long-term reliability).

2.4 Sustainability as a Multi-Dimensional Concept

The paper treats sustainability not as a vague environmental notion but as a rigorous, multi-dimensional quality attribute. It draws on the **Karlskrona Manifesto** (a foundational software engineering sustainability framework) to define four dimensions:

Dimension	What It Means for an ML System
Technical	Prediction accuracy, maintainability, system reliability
Environmental	Energy consumption, carbon emissions, hardware utilization
Economic	Compute costs, storage costs, operational overhead
Social	User trust, fairness, long-term societal benefit

True sustainability requires balancing *all four* simultaneously. This is the challenge HarmonE is built to address.

2.5 Self-Adaptive Systems and the MAPE-K Loop

A **self-adaptive system** is one that can monitor itself and its environment, reason about what it observes, and take corrective action — all without human intervention. This is sometimes called *autonomic computing*.

The dominant architectural pattern for self-adaptive systems is the **MAPE-K loop**, introduced by IBM in the early 2000s. MAPE stands for:

Phase	Role
Monitor	Continuously observe the system’s metrics (accuracy, energy, data distribution, etc.)
Analyze	Compare observed metrics against desired goals; detect violations or uncertainties
Plan	Select an appropriate adaptation strategy to correct the violation
Execute	Carry out the chosen strategy (e.g., switch model, re-train, restore archived model)
Knowledge (K)	A shared repository of goals, thresholds, historical data, and tactics that informs all four phases

Analogy / Intuition

Think of MAPE-K like a thermostat: it *monitors* temperature, *analyzes* whether it is too hot or cold relative to a target, *plans* whether to turn heating or cooling

on, and *executes* the switch. The target temperature and the allowed range are the “Knowledge.” HarmonE is a sophisticated thermostat for an ML system’s health.

2.6 KL Divergence: Measuring Data Shift

To detect whether incoming data has drifted away from the training distribution, the paper uses **KL Divergence** (Kullback-Leibler Divergence), a statistical measure of how different two probability distributions are from each other.

If P_t is the distribution of recently observed data and P_r is the reference (training) distribution, a high KL divergence $\mathcal{D}(P_t||P_r)$ signals that the data has shifted significantly and the model may no longer be well-calibrated.

2.7 Exponential Moving Average (EMA)

Rather than reacting to individual noisy measurements, it is wiser to track a *smoothed trend*. The **Exponential Moving Average** does this by weighting recent observations more heavily than older ones:

$$\text{EMA}(S_i) = \gamma S_i + (1 - \gamma) \text{EMA}(S_{i-1})$$

where $\gamma \in [0, 1]$ is a smoothing factor. A high γ makes the EMA react quickly to recent values; a low γ makes it more sluggish and stable.

3 Motivation: Why HarmonE Is Needed

3.1 The Gap in Existing Approaches

Before HarmonE, the landscape of related work had several isolated solutions, none of which addressed the full problem:

- **Standard MLOps frameworks** (e.g., periodic retraining pipelines) address technical reliability but ignore environmental and economic costs.
- **Green AI research** proposes energy-efficient model architectures, but these are static design-time solutions — they do not adapt at runtime.
- **Self-adaptive ML systems** (e.g., QoS-aware model switching) respond to runtime uncertainties but are limited to inference-time decisions and do not consider the full lifecycle or energy cost.

- **EcoMLS** introduced energy-aware switching, but only at inference time, without considering training costs or long-term lifecycle sustainability.
- **Architectural sustainability frameworks** (e.g., Decision Maps) provide design-time guidance but lack runtime integration.

The Core Gap

No existing approach provides an *end-to-end, runtime-adaptive, multi-dimensional* sustainability framework for MLOps pipelines. This is the gap HarmonE fills.

3.2 The Problem with Instantaneous Reactions

A critical insight in the paper is that existing self-adaptive systems react *immediately* to any deviation from a threshold. This is overly aggressive: a single bad interval should not trigger an expensive retraining. HarmonE instead introduces a **control-theoretic mechanism** where the threshold itself *evolves* based on the cumulative history of deviations. This means:

- Temporary spikes are tolerated.
- Only persistent, cumulative violations trigger adaptation.
- This avoids thrashing (constantly switching or retraining unnecessarily).

3.3 The Real-World Trigger: ITS Digital Twin

The motivation is grounded in a concrete collaborative project between IIIT Hyderabad and Middlesex University (London) called the **DigIT project**. The goal is to build a Digital Twin (DT) of an Intelligent Transportation System (ITS) for urban mobility.

A key component of this DT is a **traffic flow prediction layer** — an ML model that predicts how many vehicles will pass a given point in the next 5-minute window. These predictions drive:

- Adaptive traffic re-routing.
- Congestion management.
- Real-time simulation of traffic scenarios.

Traffic patterns are highly non-stationary (they change by time of day, day of week, season, accidents, events, etc.). This means the prediction model must be updated frequently — but doing so aggressively is costly. HarmonE was developed specifically to solve this tension.

4 HarmonE: The Proposed Solution

HarmonE is an **architectural approach** (not just an algorithm) that embeds self-adaptive capabilities into an MLOps pipeline using the MAPE-K loop. Its design is guided by three principles:

1. **Goal-driven adaptation:** Sustainability goals are explicitly defined at design time and drive all runtime decisions.
2. **Deliberate, not reactive:** Adaptations are triggered by cumulative deviations, not instantaneous violations.
3. **Full lifecycle coverage:** The framework manages training, inference, model versioning, and model selection — not just one phase.

HarmonE has three main architectural components: the **Managed System**, the **Managing System**, and the **Decision Map**.

4.1 Component 1: The Managed System

The Managed System represents the operational ML pipeline — the part being managed. It has two subsystems:

4.1.1 Training Subsystem

This subsystem handles all model training and retraining. It is *not autonomous* — it acts only when explicitly instructed by the Managing System. When triggered:

1. It retrains the model using current data.
2. It forwards the newly trained model and its associated training data to the Managing System for archiving.
3. This ensures that every model version is paired with the data distribution it was trained on, enabling intelligent reuse later.

4.1.2 Inference Subsystem

This subsystem makes real-time predictions. At any given moment, it uses one specific model selected by the Managing System. It logs:

- Prediction outputs.
- Ground truth values (when available).
- Inference time.

- Energy consumption.

These logs feed back into the Managing System for ongoing monitoring.

4.2 Component 2: The Decision Map (Design-Time Artifact)

The **Decision Map (DM)** is a visual and formal design-time artifact created by the system architect before deployment. It captures:

- Key **system functionalities** (what the system does).
- Associated **sustainability concerns** across all four dimensions.
- **Relationships** between functionalities and concerns (positive, negative, or undecided).
- **Adaptation boundaries** — threshold values that define acceptable operating ranges.

For the traffic prediction use case, the DM identifies:

Functionality	Positive Sustainability Impact	Negative Sustainability Impact
Model Retraining	Improves accuracy (Technical, Social)	Increases energy (Environmental) and cost (Economic)
Model Switching	Enables energy efficiency; avoids unnecessary retraining	Can reduce accuracy if lighter model is insufficient
Data & Model Versioning	Enables model reuse; improves maintainability	Increases storage costs (Economic)

The DM translates these concerns into measurable thresholds:

- Maximum allowable KL divergence before triggering retraining.
- Reference energy level $\bar{E}_{\text{ref}}$ that the system should stay near.
- Minimum acceptable performance score S_{min} .
- Maximum allowable storage for versioned models.

These boundaries are stored in the Knowledge Base and drive all runtime adaptation.

4.3 Component 3: The Managing System

The Managing System implements the MAPE-K loop and is composed of several sub-components.

4.3.1 The Knowledge Base

The Knowledge Base is the “brain” of the Managing System. It has five repositories:

1. **Data Repository:** Stores incoming sensor data and corresponding model predictions. Used for drift detection and performance assessment.
2. **Sustainability Goals Repository:** Stores the thresholds derived from the DM (e.g., $S_{\min}$, $\bar{E}_{\text{ref}}$, τ_{drift}).
3. **Tactics Repository:** A library of predefined adaptation strategies (model switch, retrain, compress, etc.) linked to specific uncertainty types.
4. **Current Model Repository:** A curated set of n candidate models $\mathcal{M} = \{M_1, M_2, \dots, M_n\}$ available for deployment. Each model represents a different accuracy-efficiency trade-off (in the paper: Linear Regression, SVM, LSTM).
5. **Versioned Model Repository (VMR):** Archives retrained models alongside the data distributions they were trained on. This enables the system to match new data distributions to historical ones and reuse past models instead of retraining from scratch.

4.3.2 Monitor: Observing the System

The Monitor component periodically collects runtime metrics. At each monitoring interval i , it computes:

Step 1 — Performance Score

Two raw metrics are combined into a single composite score:

- A_i : Prediction accuracy (e.g., R^2 score for regression tasks).
- $\bar{E}_i$: Normalized energy consumption — the current interval’s energy divided by the maximum energy seen during training. This ensures comparability across intervals.

Performance Score Formula

$$S_i = \beta A_i + (1 - \beta)(1 - \bar{E}_i)$$

where $\beta \in [0, 1]$ is a design-time parameter that weights accuracy versus energy efficiency. A value of $\beta = 0.7$, for example, would prioritize accuracy at 70% and energy efficiency at 30%.

Note that $(1 - \bar{E}_i)$ is used for energy so that *lower energy consumption contributes positively* to the score. A higher S_i means the system is performing better.

Step 2 — Smoothing via EMA

To avoid reacting to random fluctuations, the system tracks a smoothed version:

Exponential Moving Average of Performance

$$\text{EMA}(S_i) = \gamma S_i + (1 - \gamma) \text{EMA}(S_{i-1})$$

where γ is a design-time smoothing factor.

Step 3 — Drift Detection

For distributional shift, KL Divergence is computed between:

- P_t : The distribution of recently observed true values.
- P_r : The reference distribution (from training data).

All three metrics ($\text{EMA}(S_i)$, $\bar{E}_i$, $\mathcal{D}$) are passed to the Analyzer.

4.3.3 Analyzer: Detecting Uncertainty

The Analyzer applies the following three checks:

Check 1 — Performance Degradation

If $\text{EMA}(S_i) < S_{\min}$, the system’s combined accuracy-energy performance has fallen below the acceptable threshold. The Analyzer flags this and invokes the Planner.

Check 2 — Energy Violation (with Dynamic Threshold)

This is HarmonE’s most distinctive mechanism. Rather than comparing against a fixed energy threshold, the system uses a *dynamic threshold* τ_{E_i} that evolves over time:

Dynamic Energy Threshold Update Rule

$$\tau_{E_{i+1}} = \tau_{E_i} + \delta \cdot (\bar{E}_{\text{ref}} - \bar{E}_i)$$

where:

- $\bar{E}_{\text{ref}}$ is the design-time reference energy level.
- $\delta \in (0, 1)$ is a decay factor.
- $(\bar{E}_{\text{ref}} - \bar{E}_i)$ is the deviation from the reference level.

The intuition behind this rule is elegant:

- If $\bar{E}_i > \bar{E}_{\text{ref}}$ (using more energy than ideal): the deviation is *negative*, so $\tau_{E_{i+1}} < \tau_{E_i}$. The threshold *tightens* — the system becomes stricter about energy use in the next interval.

- If $\bar{E}_i < \bar{E}_{\text{ref}}$ (using less energy than ideal): the deviation is *positive*, so $\tau_{E_{i+1}} > \tau_{E_i}$. The threshold *loosens* — the system gives itself more room to tolerate a future spike.

If $\bar{E}_i > \tau_{E_i}$ at any interval, the Analyzer flags an energy uncertainty.

Analogy / Intuition

Think of a personal budget. If you overspend this month, next month’s budget is tighter. If you underspent, you carry over some room for next month. HarmonE applies this logic to energy budgeting for ML inference.

Check 3 — Data Drift

If $\mathcal{D} > \tau_{\text{drift}}$ (KL divergence exceeds the drift threshold set in the DM), a distributional shift is detected and the Planner is invoked.

4.3.4 Planner: Selecting an Adaptation Strategy

The Planner (specifically, its **Strategy Evaluator**) selects the best adaptation tactic from the Tactics Repository. The available tactics are organized by uncertainty type:

Uncertainty	Tactics Available	Strategy
Model Drift	Model Switch, Retrain	Switch to better-fit model from VMR; or retrain via incremental/transfer/full retraining
Data Drift	Model Switch, Retrain	Switch to better-fit model from VMR; or retrain with resampled/augmented data
High Energy Use	Model Switch, Model Compression	Switch to a lighter model; or quantize the current model
Performance Degradation	Model Switch, Retrain	Switch to higher-accuracy model; or retrain

How model selection works (performance-based adaptation):

The Strategy Evaluator retrieves each model’s historical EMA scores from the Knowledge Base and selects the model (excluding the currently active one) with the highest $\text{EMA}(S_i)$.

An ϵ -greedy exploration strategy is used: with probability ϵ , the Evaluator randomly picks a model from $\mathcal{M}$ instead of the best-known one. This allows the system to occasionally explore and prevents it from becoming locked into a suboptimal model.

How model selection works (drift-based adaptation):

When drift is detected, the Planner compares the current data distribution against the stored training distributions of all versioned models in the VMR. If a sufficiently similar past distribution is found, the system switches to that archived model — avoiding the cost of retraining. If no match is found, the Executor is called to perform full retraining.

4.3.5 Executor: Carrying Out the Adaptation

The Executor enacts whatever strategy the Planner selected:

- **Model switch:** The Executor updates the Inference Subsystem to use the selected model. All future predictions use the new model.
- **Retraining:** The Executor invokes the Training Subsystem. After training completes, the new model and its training data are archived in the VMR, and the Inference Subsystem is updated.
- **Version reuse:** The Executor loads a matching versioned model from the VMR and deploys it to the Inference Subsystem.

5 The Use Case: Traffic Flow Prediction in a Digital Twin

5.1 Setting

The paper validates HarmonE within a DT of an ITS. The specific task is predicting **traffic flow** (vehicles per 5-minute window) at a single sensor node in California’s highway network, using data from the **PeMS** (Performance Measurement System) dataset.

The data is structured as a supervised time-series problem:

- **Input:** Previous 5 timesteps (25 minutes of sensor readings).
- **Output:** Traffic flow at the 6th timestep (the 30th minute).

5.2 Model Spectrum

HarmonE requires a **spectrum of candidate models** with different accuracy-efficiency trade-offs. Three are used:

Model	Accuracy	Energy Cost	Speed
Linear Regression (LR)	Lowest ($R^2 \approx 0.76$)	Very low (~ 12.1 mJ)	Fastest (~ 0.83 ms)
Support Vector Machine (SVM)	Moderate ($R^2 \approx 0.79$ – 0.82)	Low (~ 14.8 – 16.3 mJ)	Fast (~ 1.1 ms)
LSTM	Highest ($R^2 \approx 0.90$)	Very high (~ 33 – 45 mJ)	Slowest (~ 3.8 ms)

5.3 Drift Induction

To test HarmonE’s adaptation capabilities under real-world conditions, the researchers *artificially introduced drift* into the test dataset. This was done using a scale-and-shift transformation applied to specific segments of the test data — simulating events like road closures or accidents that abruptly change traffic patterns.

Crucially, the same transformation was applied *twice*, creating two drift events. The second drift realigns the data with a previously seen distribution, allowing HarmonE to demonstrate *model reuse from the VMR* rather than retraining from scratch.

6 Experimental Design and Baselines

6.1 Dataset Split

- **Training set:** 1,200 samples (80%)
- **Validation set:** 240 samples (20%)
- **Test set:** 14,500 samples (held out for inference/evaluation)

6.2 Baselines

HarmonE is compared against 8 baselines representing different operational strategies:

Baseline	Description
LR	Static Linear Regression at inference; never retrained
LR+PRT	Linear Regression with Periodic Retraining (every 3,200 timesteps, regardless of need)
SVM	Static SVM at inference
SVM+PRT	SVM with Periodic Retraining
LSTM	Static LSTM at inference
LSTM+PRT	LSTM with Periodic Retraining
Switch	Switches among LR/SVM/LSTM based on detected uncertainties, but never retrains
Switch+PRT	Switch strategy combined with Periodic Retraining
HarmonE	The proposed approach

6.3 Metrics

- **Predictive Accuracy:** R^2 score (coefficient of determination); higher is better.
- **Energy Consumption:** Measured using pyRAPL, a Python library for hardware-level energy profiling on Intel processors. Reported in millijoules (mJ).
- **Inference Time:** Average time per prediction in milliseconds (ms).
- **Number of Adaptations:** How many switching, retraining, or versioning events occurred.

Each experiment was run **5 independent times** (standard practice in empirical software engineering) and results were averaged. A **20-minute cooldown period** between runs prevented hardware thermal effects from biasing energy measurements.

7 Results and Analysis

7.1 RQ1: Long-Term Environmental Sustainability

Question: How does HarmonE ensure the long-term environmental sustainability of MLS?

Finding: HarmonE consistently keeps its cumulative energy consumption *below the reference energy level* defined in the Decision Map. It achieves this by retraining only when drift is genuinely detected, rather than on a fixed schedule.

Key Result

HarmonE consumes **20.62 mJ** on average, representing:

- A **54.5% reduction** compared to LSTM+PRT (45.35 mJ) — the most accurate periodic baseline.
- An **18.6% reduction** compared to Switch+PRT (25.32 mJ) — the best non-HarmonE adaptive baseline.

The key insight is that all PRT (Periodic Retraining) approaches waste energy retraining at fixed intervals even when the model is performing well. HarmonE triggers retraining only when drift is detected, avoiding this waste.

7.2 RQ2: Energy vs. Accuracy Trade-Off

Question: How does HarmonE manage the trade-off between energy consumption and predictive accuracy?

The tension here is clear from the baselines:

- **LR and SVM** are energy-efficient but inaccurate ($R^2 = 0.76\text{--}0.82$).
- **LSTM** is highly accurate ($R^2 = 0.90$) but very expensive (32–45 mJ, 3.8 ms latency).
- **Switch (no retraining)** tries to balance but defaults to LSTM for accuracy, resulting in high energy without achieving top accuracy ($R^2 = 0.84$).

Approach	Energy (mJ)	R^2	Inf. Time (ms)	Adaptations
LR	12.12	0.7597	0.83	—
LR+PRT	12.70	0.7624	—	4
SVM	14.76	0.7897	1.11	—
SVM+PRT	16.33	0.8167	—	4
LSTM	32.91	0.9005	3.85	—
LSTM+PRT	45.35	0.9085	—	4
Switch	24.32	0.8445	2.27	13
Switch+PRT	25.32	0.8435	—	17
HarmonE	20.62	0.8628	1.89	12

Key Result

HarmonE achieves:

- **95.0%** of LSTM+PRT’s accuracy ($R^2 = 0.8628$ vs. 0.9085).
- At only **45.5%** of LSTM+PRT’s energy cost (20.62 mJ vs. 45.35 mJ).
- **53.4% faster** inference than LSTM+PRT (1.89 ms vs. 4.05 ms).

It does this while adapting 12 times — fewer than Switch+PRT (17 adaptations) and more targeted than PRT approaches (which blindly retrain 4 times).

7.3 RQ3: Efficiency of Adaptation Decisions

Question: How efficient are the adaptation decisions made by HarmonE in terms of frequency and resource usage?

This question examines whether the MAPE-K loop itself imposes significant overhead.

Key Result

- The MAPE-K loop executes in an average of **17.899 ms per invocation** — negligible relative to the inference window.
- The energy consumed by the MAPE-K loop itself is only **1.366%** of the system’s total energy usage.
- HarmonE performs **12 adaptations** per run — fewer than the Switch baseline (13) despite being more capable (it includes retraining and VMR access).

This demonstrates that the intelligence of HarmonE comes at essentially zero runtime cost.

8 Threats to Validity

The authors honestly acknowledge three categories of threats:

8.1 Internal Validity

Threat: Energy measurements and performance metrics may be affected by background OS processes or hardware variability between runs.

Mitigation: Each experiment was run 5 times independently, results were averaged, and a 20-minute cooldown was enforced between runs to stabilize hardware state.

8.2 Construct Validity

Threat: The energy measurements use the pyRAPL library, which is specific to Intel processors. Absolute values might differ on other hardware platforms.

Mitigation: The authors argue that relative trends — which is what HarmonE’s decisions depend on — should remain consistent across platforms.

8.3 External Validity

Threat: The experiments use a single domain (traffic flow prediction) and a limited set of models. Results may not generalize to computer vision, NLP, or other ML domains.

Mitigation: HarmonE is designed to be domain-agnostic; the metrics, thresholds, and tactics can be redefined for any domain. The model spectrum can be replaced with domain-appropriate alternatives.

9 Related Work: Where HarmonE Fits

Approach	Strengths	Limitations	vs. HarmonE
Standard MLOps (Amershi et al.)	Automates ML lifecycle	Ignores environmental/economic sustainability	HarmonE adds multi-dim. sustainability
Green AI (Han et al., Cai et al.)	Energy-efficient architectures	Static; no runtime adaptation	HarmonE is dynamic
EcoMLS (Tedla et al.)	Energy-aware inference switching	Only inference; no training lifecycle	HarmonE covers full lifecycle
QoS-aware switching (Kulkarni et al.)	Runtime model switching via MAPE-K	No energy/retraining cost consideration	HarmonE adds energy + drift adaptation
Decision Maps (Lago)	Architectural sustainability design	No runtime integration	HarmonE operationalizes DMs at runtime

10 The Big Picture: How Everything Connects

To consolidate the full paper into a coherent narrative:

1. **Problem:** ML systems deployed in dynamic environments degrade over time. Fixing this through frequent retraining is costly (energy, money, time). Existing solutions address only one dimension of the problem at a time.
2. **Insight:** What is needed is a framework that (a) monitors the system holistically, (b) reacts deliberately rather than impulsively, (c) leverages historical models intelligently,

and (d) balances all four sustainability dimensions.

3. **Solution — Design Time:** The system architect defines sustainability goals and trade-offs in a Decision Map. This becomes the “constitution” that governs all runtime decisions.
4. **Solution — Runtime:** The MAPE-K loop runs continuously. The Monitor aggregates accuracy and energy metrics. The Analyzer checks these against dynamic, evolving thresholds. The Planner selects the cheapest corrective action (switch model, reuse archived model, or retrain). The Executor implements it.
5. **Key Innovation:** The dynamic energy threshold (Equation 3) allows the system to accumulate “credit” from periods of good performance and “debt” from overuse — making adaptation deliberate and context-aware rather than instantaneous and aggressive.
6. **Validation:** On 14,500 timesteps of real traffic data, HarmonE achieves near-LSTM accuracy at less than half the energy cost, with negligible overhead from the adaptation mechanism itself.

11 Conclusion and Future Directions

11.1 What Was Achieved

HarmonE demonstrates that it is possible to build an ML system that genuinely manages its own sustainability:

- 54.5% energy reduction vs. the most accurate competing approach.
- 95% of LSTM+PRT’s accuracy retained.
- 53.4% faster inference than LSTM+PRT.
- MAPE-K overhead of only 1.37% of total system energy.
- Intelligent reuse of versioned models when the same drift pattern recurs.

11.2 Future Work

The authors outline the following directions:

- **Broader domain validation:** Test HarmonE in computer vision and NLP settings, where model switching and retraining have different cost profiles.
- **LLM/SLM applicability:** Apply HarmonE to Large Language Model systems, where fine-tuning costs are enormous and smart adaptation is even more critical.

- **Extended sustainability dimensions:** Investigate fairness and equity (social sustainability) more rigorously.
- **Multi-node deployment:** Extend to distributed inference scenarios where energy budgets must be allocated across multiple hardware nodes.

Final Takeaway

HarmonE is not just an engineering solution to a technical problem. It represents a shift in *philosophy*: treating sustainability as a first-class design concern in ML systems, and building the infrastructure to enforce it automatically at runtime. Its contribution is both conceptual (the integration of Decision Maps with MAPE-K for MLOps) and practical (a validated, open-source implementation that demonstrably reduces energy use without sacrificing accuracy).

Source code and data: <https://github.com/sa4s-serc/HarmonE>

Dataset: California PeMS — <https://pems.dot.ca.gov>

Original paper: arXiv:2505.13693v1 [cs.SE], May 2025